

Parcial 5

Total de puntos 520/620

Todas las preguntas que pude meter al forms, faltan un par. pero sirve.

¿Cuál es la diferencia entre el peor caso y el caso promedio en el análisis de algoritmos?

10/10

El peor caso es la con guración de datos de entrada más favorable para el algoritmo, mientras que el caso promedio describe una con guración aleatoria de datos (no pensada ni para favorecer ni para desfavorecer al algoritmo)

El peor caso es la con guración de datos de entrada más favorable para el algoritmo, mientras que el caso promedio describe una con guración de datos pensada para desfavorecer al algoritmo.

El peor caso es la con guración de datos de entrada más desfavorable para el algoritmo, mientras que el caso promedio describe una con guración aleatoria de datos (no pensada ni para favorecer ni para desfavorecer al algoritmo).

El peor caso es la con guración de datos de entrada más desfavorable para el algoritmo, mientras que el caso promedio describe una con guración de datos pensada para favorecer al algoritmo.



¿Cuáles de las siguientes son características correctas del algoritmo Shellsort? (Más de una puede ser cierta... marque TODAS las que considere válidas) 10/10

En el caso promedio, el algoritmo Shellsort es tan eficiente como el Heapsort o el Quicksort, con tiempo de ejecución $O(n \log(n))$.

El algoritmo Shellsort es complejo de analizar para determinar su rendimiento en forma matemática. Se sabe que para la serie de incrementos decrecientes usada en la implementación vista en las clases de la asignatura, tiene un tiempo de ejecución para el peor caso de $O(n^{1,5})$.

El algoritmo Shellsort consiste en una mejora del algoritmo de Selección Directa, consistente en buscar iterativamente el menor (o el mayor) entre los elementos que quedan en el vector, para llevarlo a su posición correcta, pero de forma que la búsqueda del menor en cada vuelta se haga en tiempo logarítmico.

El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.

10/10

¿Por qué es considerada una *mala idea* tomar como pivot al *primer elemento* (o al *último*) de cada partición al implementar el algoritmo Quicksort?

Seleccione una:

- ☐ a. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese ya ordenado o casi ordenado.
- ☐ b. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese completamente desordenado.
- ☐ c. No es cierto que sea una mala idea. Ambas alternativas son tan buenas como cualquier otra.
- ☐ d. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el tamaño n del arreglo fuese muy grande.

a

b

c

d



¿Cuál de las siguientes situaciones haría que el algoritmo Quicksort degenerare en su peor caso en cuanto al tiempo de ejecución, de orden n 10/10

Que el arreglo esté ya ordenado, en la misma secuencia en que se lo quiere ordenar.

El algoritmo Quicksort no tiene un peor caso $O(n)$. Su tiempo de ejecución siempre es $O(n \cdot \log(n))$.

Que el arreglo esté ya ordenado, pero al revés.

Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando

¿Cuál es la mejora esencial que el algoritmo Quicksort realiza sobre el algoritmo Bubblesort o Burbuja? 10/10

Quicksort primero determina qué tan desordenado está el arreglo, mientras que Bubblesort procede directamente a ordenarlo

Quicksort no implementa ninguna mejora sustancial sobre Bubblesort.

Quicksort acelera el cambio de posición tanto de los elementos menores como de los mayores, mientras que Bubblesort solo acelera a los mayores (o a los menores, dependiendo de la forma de implementación).

En las versiones analizadas en clases, Quicksort sólo usa un ciclo para recorrer el arreglo, mientras que Bubblesort usa dos.



¿Cuáles de las siguientes afirmaciones **son ciertas** en referencia a las *Estrategias de Resolución de Problemas* que se citan? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a.
La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.
- ☐ b.
El empleo de la *Recursividad* para resolver un problema no es recomendable en ningún caso, debido a la gran cantidad de recursos de memoria o de tiempo de ejecución que implica.
- ☐ c.
La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.
- ☐ d.
La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado.

A

B

C

D



¿Cuáles de las siguientes son correctas en cuanto a los tiempos de ejecución de los algoritmos de ordenamiento clásicos? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Algoritmo Heap Sort: $O(n \cdot \log(n))$ tanto para el caso promedio como para el peor caso

Algoritmo Shell Sort: $O(n^2)$ en el peor caso para la serie de incrementos decrecientes vista en clase.

Algoritmos directos o simples: $O(n^2)$ en el peor caso para todos ellos.

Algoritmo Quick Sort: $O(n \cdot \log(n))$ en el caso promedio, pero $O(n^2)$ en el peor caso.

$t = \text{fib}(6)$ ¿Cuántas veces en total la función $\text{fib}(n)$ se invoca para hacer más de una vez el mismo trabajo, SIN incluir en ese conteo a las invocaciones para obtener $\text{fib}(0)$ y $\text{fib}(1)$?

0

12

10

11



Considere el problema de las Ocho Reinas presentado en clases. Se ha10/10
indicado que se puede usar un arreglo rc de componentes, en el cual el
casillero $rc[col] = fil$ indica que la reina de la columna col está ubicada en
la fila fil . ¿Cuáles de las siguientes configuraciones para el arreglo rc
representan soluciones incorrectas para el problema de las Ocho
Reinas? (Más de una respuesta puede ser cierta, por lo que marque
todas las que considere correctas...)

$rc = [5, 3, 6, 0, 7, 1, 4, 2]$

$rc = [4, 7, 3, 0, 2, 5, 1, 6]$

$rc = [2, 0, 7, 4, 5, 1, 6, 3]$

$rc = [3, 5, 7, 0, 5, 1, 2, 4]$



¿Cuál de las siguientes estrategias de obtención del pivot *es la más recomendable* para evitar que el algoritmo *Quicksort* se degrade en su peor caso $O(n^2)$ en cuanto a su tiempo de ejecución?

Seleccione una:

- ☐ a.
En cada partición, usar como pivot al valor de la última casilla.
 - ☐ b.
En cada partición, usar como pivot al valor de la primera casilla.
 - ☒ c.
En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición).
- ¡Correcto!
- ☐ d.
En cada partición, usar como pivot al valor de la casilla central.

A

B

C

D



¿Cuáles de las siguientes son CIERTAS en relación al sistema de coordenadas de pantalla?

Seleccione una o más de una:

- ☐ a.
El origen del sistema de coordenadas se encuentra en el punto superior izquierdo de la pantalla o de la ventana que se esté usando.
¡Correcto!
- ☐ b.
Las filas de pantalla o de ventana con número de orden más bajo, se encuentran más cerca del borde superior que las filas con número de orden más alto.
¡Correcto!
- ☐ c.
Las filas de pantalla o de ventana con número de orden más alto, se encuentran más cerca del borde superior que las filas con número de orden más bajo.
- ☐ d.
El origen del sistema de coordenadas se encuentra en el punto inferior izquierdo de la pantalla o de la ventana que se esté usando.

a

b

c

d



¿Cuáles de las siguientes son CIERTAS en relación al sistema de coordenadas de pantalla?

Seleccione una o más de una:

- ☐ a.
El origen del sistema de coordenadas se encuentra en el punto superior izquierdo de la pantalla o de la ventana que se esté usando.
¡Correcto!
- ☐ b.
Las filas de pantalla o de ventana con número de orden más bajo, se encuentran más cerca del borde superior que las filas con número de orden más alto.
¡Correcto!
- ☐ c.
Las filas de pantalla o de ventana con número de orden más alto, se encuentran más cerca del borde superior que las filas con número de orden más bajo.
- ☐ d.
El origen del sistema de coordenadas se encuentra en el punto inferior izquierdo de la pantalla o de la ventana que se esté usando.

a

b

c

d



Si se realiza un análisis preciso del ordenamiento por Selección Directa^{10/10} para un arreglo de n componentes, se llega a la conclusión que ese algoritmo hará $n-1$ pasadas, con $n-1$ comparaciones en la primera, $n-2$ en la segunda, y así sucesivamente reduciendo de a 1 la cantidad de comparaciones hasta hacer sólo una comparación en la última pasada. Por lo tanto, el algoritmo hará invariablemente una cantidad total de $\frac{1}{2}(n - n)$ comparaciones. Sabiendo esto, ¿cuáles de las siguientes expresiones son correctas para describir la cantidad de comparaciones que hará el algoritmo, usando distintos tipos de notaciones? (Más de una respuesta puede ser correcta. Marque TODAS las que considere correctas)

Cantidad de comparaciones: $o(n^2)$

Cantidad de comparaciones: $O(n^2)$

Cantidad de comparaciones: $\Theta(n^2)$

Cantidad de comparaciones: $\Omega(n^2)$

¿Con qué nombre general se conoce en la Teoría de la Complejidad a un problema para el cual sólo se conocen algoritmos cuyo tiempo de ejecución es exponencial (o sea, problemas para los que todas las soluciones conocidas son algoritmos con tiempo $O(2 \text{ elevado a } n)$)?

Problemas Inmanejables

Problemas Intratables

Problemas Imperdonables

Problemas Irresolubles



	$O(\log(n))$	$O(n^2)$	$O(n^3)$	$O(n)$	$O(1)$	$O(n \cdot \log(n))$	Puntuación
--	--------------	----------	----------	--------	--------	----------------------	------------

Ordenamiento por selección directa							
--	--	--	--	--	--	--	--

Búsqueda secuencial en un arreglo (ordenado o desordenado).							
---	--	--	--	--	--	--	--

Acceso directo a un casillero de un vector.							
--	--	--	--	--	--	--	--

Ordenamiento rápido (Quicksort)							
---------------------------------------	--	--	--	--	--	--	--

Multiplicación de matrices cuadradas de tamaño $n \cdot n$							
---	--	--	--	--	--	--	--

Búsqueda binaria en un arreglo ya ordenado							
---	--	--	--	--	--	--	--



Suponga que se quiere plantear una definición recursiva del concepto de bosque. ¿Cuál de las siguientes propuestas generales es correcta y constituye la mejor definición?

Un bosque es un conjunto de árboles que puede estar vacío, o puede contener uno o más árboles agrupados con otro bosque.

Un bosque es un bosque.

Un bosque es un conjunto de árboles que puede estar vacío, o puede contener n árboles (con $n > 0$).

Un bosque es un conjunto que puede contener uno o más árboles agrupados con otro bosque.



Si los algoritmos de *ordenamiento simples* tienen todos un tiempo de ejecución $O(n^2)$ en el peor caso, entonces: ¿cómo explica que las mediciones efectivas de los tiempos de ejecución de cada uno sean diferentes frente al mismo arreglo?

Seleccione una:

- ☐ a.
La notación *Big O* rescata el término más significativo en la expresión que calcula el rendimiento, descartando constantes y otros términos que podrían no coincidir en los tres algoritmos.
- ☐ b.
Los tiempos deben coincidir. Si hay diferencias, se debe a errores en los instrumentos de medición o a un planteo incorrecto del proceso de medición.
- ☐ c.
La notación *Big O* no se debe usar para estimar el comportamiento en el peor caso, sino sólo para el caso medio.
- ☐ d.
La notación *Big O* no se usa para medir tiempos sino para contar comparaciones u otro elemento de interés. Es un error, entonces, decir que los tiempos tienen "orden n cuadrado".

a

b

c

d



Suponga que dispone de cuatro algoritmos diferentes para resolver el mismo problema, y que se sabe que los tiempos de ejecución (en el peor caso) son, respectivamente: $O(n \cdot \log(n))$, $O(n^3)$, $O(n^2)$ y $O(n)$. ¿Cuál de esos tres algoritmos debería elegir, suponiendo que todos hacen el mismo consumo razonable de memoria?

0/10

El algoritmo cuyo tiempo de ejecución es $O(n \cdot \log(n))$

El algoritmo cuyo tiempo de ejecución es $O(n^2)$

El algoritmo cuyo tiempo de ejecución es $O(n^3)$

El algoritmo cuyo tiempo de ejecución es $O(n)$

10/10

¿Cuáles de las siguientes afirmaciones son correctas respecto de las características de la recursividad? (Más de una puede ser cierta, por lo que marque todas las que considere válidas):

Seleccione una o más de una:

- ☐ a. La recursividad puede ocupar más memoria que un algoritmo iterativo equivalente.
- ☐ b. La recursividad utiliza memoria de stack.
- ☐ c. La recursividad siempre ocupa más memoria que el algoritmo iterativo equivalente.
- ☐ d. La recursividad utiliza memoria ROM.

a

b

c

d

Dado un arreglo de n componentes... ¿qué significa decir que en el peor caso la cantidad de comparaciones que realiza el algoritmo de búsqueda secuencial es $O(n)$ (o sea: del orden de n)? 10/10

Significa que en el peor caso el algoritmo hará siempre más de n comparaciones.

Significa que en el peor caso el algoritmo hará n comparaciones.

Significa que en el peor caso el algoritmo no hará ninguna comparación.

Significa que en el peor caso el algoritmo siempre hará menos de n comparaciones

¿Cuántas comparaciones en el peor caso obliga a hacer una búsqueda secuencial en una lista ordenada (o en un arreglo ordenado) que contenga n valores? 10/10

Peor caso: $O(1)$ comparaciones

Peor caso: $O(n^2)$ comparaciones.

Peor caso: $O(n)$ comparaciones.

Peor caso: $O(\log(n))$ comparaciones.

Se tiene un algoritmo que realiza cierta cantidad de procesos sobre un conjunto de n datos y un minucioso análisis matemático ha determinado que la cantidad de procesos que el algoritmo realiza en el peor caso viene descrito por la función $f(n) = 3n^3 + 5n^2 + 2n$. ¿Cuál de las siguientes expresiones representa mejor el orden del algoritmo para el peor caso? 10/10

$O(n^3)$

$O(n^2)$

$O(3n^3 + 5n^2 + 2n)$

$O(n^3 + n^2)$

Dado un arreglo de n componentes... ¿qué significa decir que en el peor caso la cantidad de comparaciones que realiza el algoritmo de búsqueda secuencial es $O(n)$ (o sea: del orden de n)?

Significa que en el peor caso el algoritmo hará siempre más de n comparaciones.

Significa que en el peor caso el algoritmo hará n comparaciones.

Significa que en el peor caso el algoritmo no hará ninguna comparación.

Significa que en el peor caso el algoritmo siempre hará menos de n comparaciones

¿Cuántas comparaciones en el peor caso obliga a hacer una búsqueda secuencial en una lista ordenada (o en un arreglo ordenado) que contenga n valores?

Peor caso: $O(1)$ comparaciones

Peor caso: $O(n^2)$ comparaciones.

Peor caso: $O(n)$ comparaciones.

Peor caso: $O(\log(n))$ comparaciones.

Se tiene un algoritmo que realiza cierta cantidad de procesos sobre un conjunto de n datos y un minucioso análisis matemático ha determinado que la cantidad de procesos que el algoritmo realiza en el peor caso viene descrito por la función $f(n) = 3n + 5n + 2n$. ¿Cuál de las siguientes expresiones representa mejor el orden del algoritmo para el peor caso?

$O(n^3)$

$O(n^2)$

$O(3n + 5n + 2n)$

$O(n^3 + n^2)$

Considere la siguiente función (vista en clases) basada en **backtracking** para resolución del **Problema de las Ocho Reinas**, e indique cuál de las opciones que se muestran es co

```
def intend(col):
    global rc, qr, qid, qnd

    fil, res = -1, False
    while not res and fil != 7:
        res = False
        fil += 1
        di = col + fil
        dn = (col - fil) + 7
        if qr[fil] and qid[di] and qnd[dn]:
            rc[col] = fil
            qr[fil] = qid[di] = qnd[dn] = False

            if col < 7:
                res = intend(col + 1)
            if not res:
                qr[fil] = qid[di] = qnd[dn] = True
                rc[col] = -1
        else:
            res = True
    return res
```

Seleccione una:

- ☐ a. La función es correcta y funciona sin problemas.
- ☐ b. La función no es correcta porque no debe asignarse a **rc[col]** el valor de -1.
- ☐ c. La función no es correcta porque **qr** debe señalar las columnas disponibles y no las filas.
- ☐ d. La función no es correcta porque la diagonal normal debe almacenar valores (**columna + fila**).

a

b

c

d



10/10

¿Cuál de los siguientes esquemas simples de pseudocódigo representa a un algoritmo planteado mediante estrategia Divide y Vencerás, pero con tiempo de ejecución $O(n \log(n))$?

Seleccione una:

- ☐ a. `proceso(partición):`
 `adicional() [==>  $O(n^3)$ ]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☐ b. `proceso(partición):`
 `adicional() [==>  $O(n)$ ]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☐ c. `proceso(partición):`
 `adicional() [==>  $O(n^2)$ ]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☐ d. `proceso(partición):`
 `adicional() [==>  $O(1)$ ]`
 `proceso(partición/2)`
 `proceso(partición/2)`

a

b

c

d

Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(1)$?

El tiempo de ejecución es logarítmico: a medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave

El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

El tiempo de ejecución es constante, sin importar la cantidad de datos.

El tiempo de ejecución siempre es de un segundo, sin importar la cantidad de datos.



¿Cuáles de las siguientes afirmaciones son correctas en relación a conceptos elementales del análisis de algoritmos? (Más de una respuesta puede ser válida, por lo que marque todas las que considere correctas).

Seleccione una o más de una:

- ☐ a.
Los dos factores de eficiencia más comúnmente utilizados en el análisis de algoritmos son el tiempo de ejecución de un algoritmo y el espacio de memoria que un algoritmo emplea.
- ☐ b.
El análisis del *peor caso* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar la configuración más desfavorable posible de los datos que recibe.
- ☐ c.
La notación Big O se usa para expresar el rendimiento de un algoritmo en términos de una función que imponga una cota inferior para ese algoritmo en cuanto al factor medido (tiempo o espacio de memoria).
- ☐ d.
El análisis del *caso promedio* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar una configuración de datos que llegan en forma aleatoria.

a

b

c

d

¿Qué se entiende, en el contexto del Análisis de Algoritmos, por un Orden de Complejidad? 10/10

Un conjunto o familia de subrutinas con similares objetivos (equivalente al concepto de módulo).

Un conjunto o familia de funciones matemáticas que se comportan asintóticamente de la misma forma.

Un conjunto de datos ordenados.

Un conjunto o familia de algoritmos que resuelven el mismo problema.

¿Cuáles de los siguientes son factores de eficiencia comunes a considerar en el análisis de algoritmos? (Más de una respuesta puede ser válida... marque todas las que considere correctas). 10/10

El consumo de memoria

La calidad aparente de la interfaz de usuario.

El tiempo de ejecución.

La complejidad aparente del código fuente.



Considere el problema del Cambio de Monedas analizado en clases, y la solución mediante un Algoritmo Ávido también presentada en clases. ¿Cuáles de las siguientes afirmaciones son ciertas en relación al problema y al algoritmo citado? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Si el Problema de Cambio de Monedas no puede resolverse en forma óptima para un conjunto dado de monedas que incluya a la de 1 centavo, mediante el Algoritmo Ávido propuesto, entonces el problema no tiene solución.

El Algoritmo Ávido sugerido para el problema del Cambio de Monedas funciona correctamente para cualquier conjunto de valores nominales de monedas, siempre y cuando ese conjunto incluya a la moneda de 1 centavo.

El Algoritmo Ávido sugerido para el Problema del Cambio de Monedas falla si el valor x a cambiar tiene una moneda igual a x en el conjunto de valores nominales: en ese caso, el algoritmo provoca un error de runtime y se interrumpe.

Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no habrá solución posible para muchos valores de cambio.



¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(n)$?

A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.

El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

El tiempo de ejecución es constante, sin importar la cantidad de datos.

El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.

¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(\log(n))$?

El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.

El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.

El tiempo de ejecución es constante, sin importar la cantidad de datos.



Dada la siguiente función recursiva del factorial:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

La función devuelve como resultado: -24.

Así como está planteada, si entra el -4 (o cualquier otro negativo) como parámetro, la condición de corte nunca es cierta y se producirá un error de stack overflow en algún momento

La función no tiene de nido ningún caso base, que es aquel caso que se resuelve sin recursividad, y por lo tanto no está bien planteada

La función devuelve como resultado: 24.

¿Cuál de las siguientes situaciones haría que el algoritmo *Quicksort* degenerara en su peor caso en cuanto al tiempo de ejecución, de orden n^2 ?

Seleccione una:

- ☐ a.
Que el arreglo esté ya ordenado, pero al revés.
- ☐ b.
Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando.
¡Correcto!
- ☐ c.
El algoritmo Quicksort no tiene un peor caso $O(n^2)$. Su tiempo de ejecución siempre es $O(n \log(n))$.
- ☐ d.
Que el arreglo esté ya ordenado, en la misma secuencia en que se lo quiere ordenar.

a

b

c

d



¿Cuál es la cantidad de niveles del árbol de invocaciones recursivas que se genera al ejecutar el Quicksort para ordenar un arreglo de n elementos, en el **peor caso?** (Es decir: ¿Cuál es la altura de ese árbol en el peor **caso?**) 0/10

Altura = $\log(n)$

Altura = n

Altura = $n * \log(n)$

Altura = n^2

¿Cuál es la principal característica de todos los métodos de ordenamiento conocidos como métodos simples o directos?

10/10

Tienen un tiempo de ejecución de orden cuadrático.

Son muy fáciles de programar.

Son muy veloces para cualquier tamaño del arreglo a ordenar.

Tienen un tiempo de ejecución de orden lineal.



¿Cuál de las siguientes resume en forma correcta la idea general de la estrategia *Divide y Vencerás* para resolución de problemas?

Seleccione una:

- ☐ a.
El conjunto de n datos se divide en subconjuntos de aproximadamente el mismo tamaño ($n/2$, $n/3$, $n/4$, etc.). Luego se aplica recursión para procesar cada uno de esos subconjuntos. Finalmente se unen las partes que se acaban de procesar para lograr el resultado final.
- ☐ b.
Se usa una tabla para almacenar los resultados de los subproblemas que se hayan calculado, y luego cuando algún subproblema vuelve a aparecer se toma su valor desde la tabla, para evitar pérdida de tiempo.
- ☐ c.
Se aplica una regla simple que parece ser beneficiosa, sin volver atrás ni medir las consecuencias de aplicar esa regla, con la esperanza de lograr el resultado óptimo al final.

A

B

C

¿Cuál es la cantidad de niveles del árbol de invocaciones recursivas que se genera al ejecutar el Quicksort para ordenar un arreglo de n elementos, en el **caso promedio**? (Es decir: ¿Cuál es la altura de ese árbol en el caso promedio?)

Altura = $\log(n)$

Altura = n

Altura = $n * \log(n)$

Altura = n^2

Considere el problema del Cambio de Monedas analizado en clases, y la solución mediante un Algoritmo Ávido también presentada en clases. ¿Cuáles de las siguientes afirmaciones **son ciertas** en relación al problema y al algoritmo citado? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a.
Si el Problema de Cambio de Monedas no puede resolverse en forma óptima para un conjunto dado de monedas que incluya a la de 1 centavo, mediante el Algoritmo Ávido propuesto, entonces el problema no tiene solución.
- ☐ b.
El Algoritmo Ávido sugerido para el problema del Cambio de Monedas funciona correctamente para cualquier conjunto de valores nominales de monedas, siempre y cuando ese conjunto incluya a la moneda de 1 centavo.
- ☐ c.
El Algoritmo Ávido sugerido para el Problema del Cambio de Monedas falla si el valor x a cambiar tiene una moneda igual a x en el conjunto de valores nominales: en ese caso, el algoritmo provoca un error de runtime y se interrumpe.
- ☐ d.
Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no habrá solución posible para muchos valores de cambio.

☐ A

☐ B

☐ C

☐ D

Respuesta correcta



```
__author__ = 'Cátedra de AED'

def menu():
    print('1. Opcion 1')
    print('2. Opcion 2')
    print('3. Salir')
    op = int(input('Ingrese opcion: '))
    if op == 1:
        print('Eligio la opcion 1...')
    elif op == 2:
        print('Eligio la opcion 2...')
    elif op == 3:
        return
    menu()

# script principal...
menu()
```

- ☐ La versión recursiva que se mostró no funciona. Sólo permite cargar una vez la opción elegida, e inmediatamente se detiene.
- ☐ No hay ningún problema ni contraindicación. Y no sólo eso: la versión recursiva es más simple y compacta que la versión basada en un ciclo, por lo cual es incluso preferible usarla.
- ☐ Esta versión recursiva posiblemente sea más simple y compacta que la versión basada en un ciclo, pero la versión recursiva pide memoria de stack cada vez que se invoca, por lo que es más conveniente la iterativa.

Respuesta correcta



¿Por qué es considerada una *mala idea* tomar como pivot al *primer elemento* (o al *último*) de cada partición al implementar el algoritmo *Quicksort*?

Seleccione una:

- ☐ a.
No es cierto que sea una mala idea. Ambas alternativas son tan buenas como cualquier otra.
- ☐ b.
Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese completamente desordenado.
- ☐ c.
Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese ya ordenado o casi ordenado.
¡Correcto!
- ☐ d.
Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el tamaño n del arreglo fuese muy grande.

A

B

C

D



✓ ¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(n^2)$?

10/10

☐ El tiempo de ejecución es constante, sin importar la cantidad de datos.

☐ A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.

☒ El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.



☐ El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

0/10

¿Cuáles de las siguientes son ciertas respecto de la estrategia de resolución de problemas conocida como *Algoritmos Ávidos*?

Seleccione una o más de una:

- ☐ a. Se trata de un planteo basado en identificar una regla local que intuitivamente parece correcta, y aplicarla una y otra vez sin volver atrás ni analizar caminos alternativos, hasta llegar a la solución del problema global.
- ☐ b. Normalmente, una ventaja de intentar aplicar un algoritmo ávido es que si bien la validez de la regla local debe ser demostrada formalmente, eso es comúnmente fácil de hacer.
- ☐ c. Normalmente, una ventaja de un algoritmo ávido es que, si la regla local aplicada es correcta, suele ser sencillo de plantear y de comprender, además de veloz para ejecutar.
- ☐ d. Se trata de un planteo basado en explorar y analizar cada camino posible para resolver un problema, de forma que si detecta que algún camino conduce a una solución incorrecta, se abandone ese camino y se regrese para explorar otros posibles, hasta dar con la solución correcta.

a

b

c

d

Respuesta correcta



```
def procesar(n, m):  
    for i in range(n+1):  
        for j in range(m+1):  
            # ... acciones sencillas a realizar...  
            # ... suponga que no hay otro ciclo aquí...  
            # ... y que sólo aparecen operaciones de tiempo constan  
te...
```

$O(n*m)$

$O(n*n)$

$O(m)$

$O(n)$



```
def intend(col):  
    global rc, qr, qid, qnd  
  
    fil, res = -1, False  
    while not res and fil != 7:  
        res = False  
        fil += 1  
        di = col + fil  
        dn = (col - fil) + 7  
        if qr[fil] and qid[di] and qnd[dn]:  
            rc[col] = fil  
            qr[fil] = qid[di] = qnd[dn] = False  
  
        if col < 7:  
            res = intend(col + 1)  
            if not res:  
                qr[fil] = qid[di] = qnd[dn] = True  
                rc[col] = -1  
        else:  
            res = True  
  
    return res
```

Seleccione una:

- ☐ a. La recursión corta cuando res es True o fil es 7.
- ☐ b. La recursión corta cuando res es True y fil es 7.
- ☐ c. La recursión corta cuando res es False y fil es distinto 7.
- ☐ d. La recursión corta solo cuando se terminan de recorrer todas las columnas.

a

b

c

d





10/10

El producto de dos números a y b mayores o iguales cero, es en última instancia *una suma*: se puede ver como sumar b veces el número a , o como sumar a veces el número b . Por ejemplo: $5 * 3 = 5 + 5 + 5$ o bien $5 * 3 = 3 + 3 + 3 + 3 + 3$. Sabiendo esto, se puede intentar hacer una definición recursiva de la operación $\text{producto}(a, b)$ [$a \geq 0, b \geq 0$], que podría ser la que sigue:

$$\text{producto}(a, b) = \begin{cases} 0 & \text{si } a == 0 \text{ o } b == 0 \\ a + \text{producto}(a, b-1) & \text{si } a > 0 \text{ y } b > 0 \end{cases}$$

[a y b enteros,
 $a \geq 0$ y $b \geq 0$]

¿Es correcto el siguiente planteo de la función $\text{producto}(a, b)$?

```
def producto(a, b):
    if a == 0 or b == 0:
        return 0
    return a + producto(a, b-1)
```

No. La función sugerida siempre retornará 0, sean cuales fuesen los valores de a y b .

No. La última línea debería decir $\text{return } a + \text{producto}(a-1, b-1)$ en lugar de $\text{return } a + \text{producto}(a, b-1)$.

No. La propia definición previa del concepto de producto es incorrecta: un producto es una multiplicación, y no una suma...

Sí. Es correcto.



¿Cuál de las siguientes estrategias de obtención del pivot es la más recomendable para evitar que el algoritmo Quicksort se degrade en su peor caso $O(n^2)$ en cuanto a su tiempo de ejecución?

10/10

En cada partición, usar como pivot al valor de la primera casilla.

En cada partición, usar como pivot al valor de la casilla central.

En cada partición, usar como pivot al valor de la última casilla.

En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición).

10/10

```
def factorial(n):  
    f = n * factorial(n - 1)  
    if n == 0:  
        return 1  
    return f
```

bien planteada

siempre retorna -1

esta bien planteada pero con el orden incorrecto.



10/10

```
ac = 0
for i in range(10):
    ac += i
print(ac)
```

$O(1)$

$O(n^2)$

$O(n)$

$O(n^3)$



```
def get_pivot_m3(v, izq, der):  
    # calculo del pivot: mediana de tres...  
    central = int((izq + der) / 2)  
  
    if v[der] < v[izq]:  
        v[der], v[izq] = v[izq], v[der]  
  
    if v[central] < v[izq]:  
        v[central], v[izq] = v[izq], v[central]  
  
    if v[central] > v[der]:  
        v[central], v[der] = v[der], v[central]  
  
    return v[central]
```

$O(\log(n))$

$O(1)$

$O(n^2)$

$O(n)$



¿Cuáles de las siguientes propuestas generales son **ciertas** en relación al segmento de memoria conocido como *Stack Segment*?

Seleccione una o más de una:

- ☐ a.
El Stack Segment funciona como una cola (o fila) de datos (modalidad *FIFO*: First In - First Out): el primer dato en llegar, se almacena al frente del stack, y será por eso el primero en ser retirado.
- ☐ b.
El Stack Segment se utiliza como soporte interno en el proceso de invocación a funciones (*con o sin recursividad*): se va llenando a medida que se desarrolla la cascada de invocaciones, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.
¡Correcto!
- ☐ c.
El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad *LIFO*: Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado.
- ☐ d.
El Stack Segment se utiliza como soporte interno *solamente* en el proceso de invocación a funciones recursivas: se va llenando a medida que la cascada recursiva se va desarrollando, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.

a

B

C

D

¿En cuáles de las siguientes situaciones el uso de recursividad está efectivamente recomendado? (Más de una respuesta puede ser válida. Marque todas las que considere correctas) 10/10

Siempre que se pueda escribir una definición recursiva del problema.

Nunca.

Recorrido y procesamiento de estructuras de datos no lineales como árboles y grafos

Generación y procesamiento de imágenes y gráficos fractales (imágenes compuestas por versiones más simples de la misma imagen original)

Cuáles de los siguientes son conocidos algoritmos basados en la estrategia Divide y Vencerás? (Más de una respuesta puede ser correcta, por lo que marque todas las que considere válidas) 10/10

Algoritmo Quicksort para ordenamiento de arreglos

Algoritmo Heapsort para ordenamiento de arreglos.

Algoritmo Mergesort para ordenamiento de arreglos

Algoritmo Shellsort para ordenamiento de arreglos.

Google no creó ni aprobó este contenido. - [Condiciones del Servicio](#) - [Política de Privacidad](#)

Google Formularios

